KROMA

Bonjour à tous. Je m’appelle Julien Burin, candidat au titre professionnel de Développeur Web et Web Mobile. Passionné par les technologies numériques et la création d'interfaces interactives, j'ai suivi un parcours de formation rigoureux qui m'a permis d'acquérir les compétences nécessaires pour concevoir et développer des applications web complètes.

Ce dossier retrace la réalisation de mon projet de fin d'études, **KROMA**, qui synthétise l'ensemble des savoir-faire techniques et méthodologiques que j'ai acquis. Ce projet a été pour moi l'occasion de me confronter à la réalité du développement : de la compréhension du besoin jusqu'à la mise en production, en passant par la conception de la base de données et le développement d'interfaces dynamiques.

Le Projet

KROMA est une plateforme d'apprentissage en ligne basée sur la gamification (ludification). L'objectif principal de l'application est de rendre l'apprentissage plus engageant et moins monotone que les plateformes traditionnelles.

Pourquoi **KROMA** ? Dans un contexte où l'attention des utilisateurs est une ressource rare, l'apprentissage en ligne souffre souvent d'un fort taux d'abandon. **KROMA** répond à cette problématique en intégrant des mécaniques de jeu (points d'expérience, niveaux, monnaie virtuelle, badges, classements) directement dans le processus d'apprentissage.

Les objectifs du projet sont multiples :

1. **Engager l'utilisateur** : Via un système de récompenses immédiates (feedback visuel, gains de pièces/XP).
2. **Fidéliser** : Grâce à des fonctionnalités comme le "Streak" (série de jours consécutifs) et les classements hebdomadaires.
3. **Tester les connaissances** : Proposer des quiz interactifs avec différents niveaux de difficulté.

Public cible : L'application s'adresse à tout type d'apprenant souhaitant tester ses connaissances de manière ludique, qu'il s'agisse d'étudiants cherchant à réviser ou d'autodidactes.

Analyse de la Demande et du Besoin

Pour mener à bien ce projet, j'ai d'abord réalisé une analyse approfondie des besoins fonctionnels et techniques.

Le projet part d'un constat simple : il manque des outils open-source ou accessibles permettant de créer facilement des quiz gamifiés personnalisables. La demande initiale consistait à développer une application Web "Full Stack" (Frontend + Backend) capable de gérer des utilisateurs, des contenus pédagogiques (Challenges) et une progression.

J'ai identifié les fonctionnalités cœur indispensables :

Gestion des utilisateurs :

- Inscription et connexion sécurisées.
- Gestion de profil (Pseudo, adresse e-mail, mot de passe).
- Tableau de bord personnel (Pour suivre la progression).

Système de Jeu :

- Lister les quiz disponibles.
- Répondre à un quiz.
- Obtenir un résultat immédiat.

Mécaniques de Gamification :

- Gestion de la vie ("Cœurs") : chaque partie coûte une vie.
- Boutique : acheter des vies avec les pièces gagnées.
- Classement (Leaderboard) : voir sa position par rapport aux autres joueurs.

Administration :

- Pouvoir créer et modifier des quiz.

Contraintes Techniques

Pour répondre aux exigences de l'examen et du marché actuel, j'ai imposé les contraintes suivantes :

- **Application Web Responsive :** L'interface doit être parfaitement utilisable sur Mobile et Desktop.
- **Architecture Séparée :** Une API REST distincte de l'interface utilisateur pour garantir la scalabilité.
- **Sécurité des Données :** Les mots de passe doivent être hachés, et les échanges sécurisés.
- **Performance :** L'application doit être fluide et rapide, sans rechargement de page intempestif.

Veille Technologique

Avant de choisir ma stack technique, j'ai réalisé une veille pour identifier les outils les plus pertinents :

- **Pour le Frontend :** React s'est imposé pour sa popularité en entreprise et son écosystème riche. J'ai choisi Vite comme outil de build pour sa rapidité supérieure à Create-React-App.
- **Pour le Backend :** Node.js avec Express est un standard. Pour la base de données, l'utilisation d'un ORM comme Prisma avec PostgreSQL permet une gestion rigoureuse des données relationnelles tout en offrant une excellente expérience développeur.

Cette phase d'analyse m'a permis de structurer ma pensée et de définir une feuille de route claire pour le développement.

Gestion de Projet

Pour organiser mon travail, j'ai adopté une approche inspirée des méthodes Agiles. Bien que travaillant seul sur la phase de développement, cette organisation est essentielle pour maintenir un rythme soutenu et une direction claire.

J'ai découpé le projet en plusieurs itérations d'une semaine environ, avec des objectifs précis à chaque étape :

- **Sprint 1 : Conception & Base de données** (MCD, Maquettes, Initialisation Repo).
- **Sprint 2 : Backend Core** (Auth, CRUD de base Users/Quiz).
- **Sprint 3 : Frontend Core** (Mise en place UI, Connexion API, Routing).
- **Sprint 4 : Features Avancées & Polissage** (Gamification, Shop, Animations).

J'ai utilisé un tableau de type Kanban pour suivre mes tâches avec les colonnes classiques : À faire, En cours, Terminé.

J'ai identifié le chemin critique : la base de données devait être solide avant de commencer le Backend, et l'API devait être fonctionnelle avant de lier le Frontend.

Conception Fonctionnelle

Avant d'écrire la moindre ligne de code, j'ai défini précisément ce que l'application allait faire et comment l'utilisateur allait interagir avec elle.

Les principaux cas d'utilisation identifiés sont :

Visiteur : Se créer un compte, Se connecter.

Utilisateur Connecté :

- Consulter son profil et ses stats.
- Lancer un challenge (Quiz).
- Parcourir la boutique.
- Suivre d'autres utilisateurs.

Administrateur : Créer/Modifier/Supprimer des challenges.

Pour se faire, j'ai opté pour une architecture Client-Serveur découplée :

- **Serveur (API) :** Héberge la logique métier et l'accès aux données. Il expose des points d'entrée RESTful.
- **Client (SPA) :** Une application React qui consomme ces données et gère l'affichage dynamique.

Cette séparation permet de faire évoluer le Frontend (par exemple vers une appli mobile native React Native) sans toucher au Backend.

Modélisation des Données

La base de données est la colonne vertébrale de l'application. J'ai conçu un Modèle Conceptuel de Données (MCD) robuste pour gérer les relations complexes de la gamification.

Voici les entités principales de mon schéma (implémenté via Prisma) :

- **User :** L'utilisateur central. Il possède des attributs de jeu (xp, coins, hearts) et des relations vers ses activités.
- **Account :** Contient les informations de connexion, la plateforme notamment.
- **Session :** Contient les informations de session utilisateur.
- **Challenge :** Représente un quiz. Il a une Difficulty EASY, MEDIUM, HARD et contient plusieurs Question.
- **Question :** Contient les choix, la réponse et l'ID du Challenge.
- **UserChallenge (Table de liaison) :** C'est l'historique. Elle enregistre qu'un utilisateur X a terminé le challenge Y à la date T. C'est essentiel pour ne pas donner l'XP deux fois pour le même quiz.
- **Follow :** Une relation réflexive sur la table User pour gérer les abonnements, comme sur un réseau social.

Choix de PostgreSQL : J'ai choisi une base de données relationnelle (SQL) plutôt que NoSQL car mes données sont très structurées et fortement liées (un utilisateur à des scores, suit des amis, possède des badges). L'intégrité des données est primordiale ici.

User		
String	id	Primary Key
String	email	Unique Key
String	name	
Enum	role	USER | ADMIN
Int	xp	
Int	coins	
Int	hearts	
String[]	badges	
Int	streak	
int	completedChallengeCount	

Challenge		
String	id	Primary Key
String	title	
Enum	difficulty	EASY | MEDIUM | HARD
Int	xp	
Int	coins	
String []	tags	

UserChallenge		
String	id	Primary Key
String	userId	Foreign Key
String	challengeId	Foreign Key
Date	completedAt	

Questions		
String	id	Primary Key
String	challengeId	Foreign Key
String	question	
String []	options	
Int	answer	

Account		
String	id	Primary Key
String	userId	Foreign Key
String	providerId	
String	accountId	

Session		
String	id	Primary Key
String	userId	Foreign Key
String	token	Unique Key
Date	expiresAt	

Follow		
String	followerId	Foreign Key
String	followingId	Foreign Key

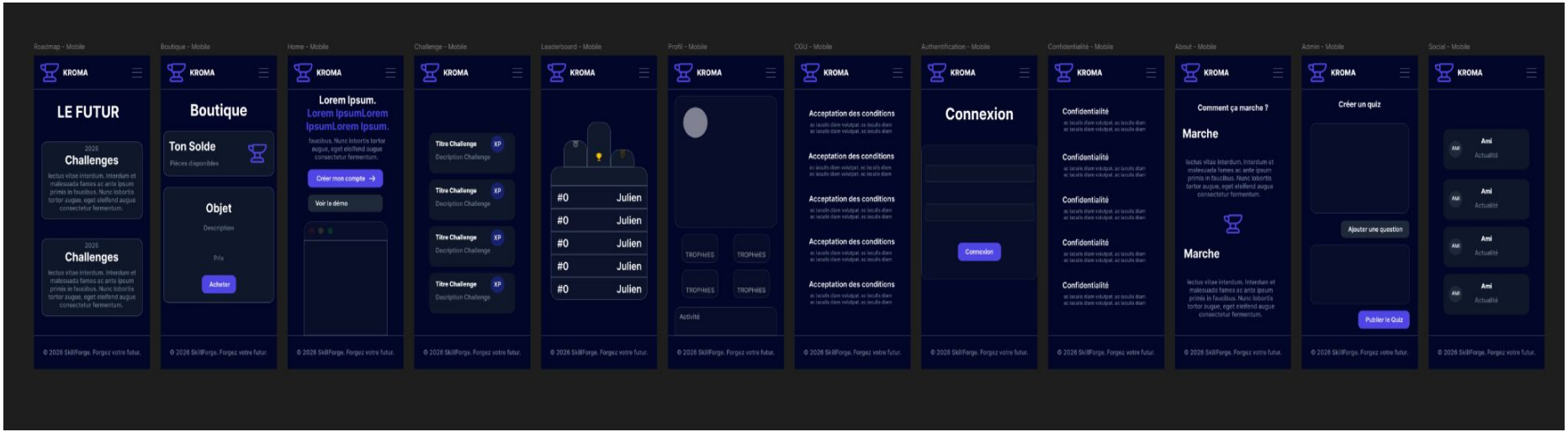
Maquettage

Avant le développement, j'ai réalisé des maquettes fonctionnelles, puis graphiques. L'objectif était de valider l'ergonomie.

- **Choix des couleurs :** Une palette vibrante (Violet/Indigo) pour rappeler l'univers du jeu vidéo et du dynamisme.
- **Typographie :** Une police sans-serif moderne (Inter) pour une lisibilité optimale sur écran.
- **Composants :** J'ai conçu une interface modulaire (Cartes pour les quiz, Boutons d'action clairs, Barres de navigation fixes sur mobile).

Cette phase de conception m'a fait gagner un temps précieux lors de l'intégration CSS, car je n'avais plus à hésiter sur les tailles ou les couleurs : tout était décidé.

Confidentialité - PC
Leaderboard - PC
CGU - PC
Profil - PC
About - PC
Home - PC
Authentification - PC
Boutique - PC
Social - PC
Challenge - PC
Roadmap - PC
Admin - PC
KROMA
Roadmap Fonctionnement Classement Défis Social Boutique
Connexion
Inscription
Politique de confidentialité
Leaderboard
Acceptation des conditions
Comment ça marche ?
Marche
Lorem Ipsum.
Lorem IpsumLorem IpsumLorem Ipsum.
Créer mon compte
Voir la démo
Connexion
Boutique
Ton Solde
Objet
Acheter
Fil d'actualité
Ami
Challenges
Tous
Non complétés
LE FUTUR
Créer un quiz
Ajouter une question
Publier le Quiz
Flow 1
© 2026 SkillForge. Forgez votre futur.
Conditions d'utilisation
Confidentialité


Roadmap - Mobile
KROMA
LE FUTUR
2026
Challenges
lectus vitae interdum. Interdum et malesuada fames ac ante ipsum primis in faucibus. Nunc lobortis tortor augue, eget eleifend augue consectetur fermentum.
2026
Challenges
lectus vitae interdum. Interdum et malesuada fames ac ante ipsum primis in faucibus. Nunc lobortis tortor augue, eget eleifend augue consectetur fermentum.
© 2026 SkillForge. Forgez votre futur.
Boutique - Mobile
KROMA
Boutique
Ton Solde
Pièces disponibles
Objet
Description
Prix
Acheter
© 2026 SkillForge. Forgez votre futur.
Home - Mobile
KROMA
Lorem Ipsum.
Lorem IpsumLorem IpsumLorem Ipsum.
faucibus. Nunc lobortis tortor augue, eget eleifend augue consectetur fermentum.
Créer mon compte →
Voir la démo
© 2026 SkillForge. Forgez votre futur.
Challenge - Mobile
KROMA
Titre Challenge
XP
Decription Challenge
Titre Challenge
XP
Decription Challenge
Titre Challenge
XP
Decription Challenge
Titre Challenge
XP
Decription Challenge
© 2026 SkillForge. Forgez votre futur.
Leaderboard - Mobile
KROMA
#0 Julien
#0 Julien
#0 Julien
#0 Julien
#0 Julien
© 2026 SkillForge. Forgez votre futur.
Profil - Mobile
KROMA
TROPHÉES
TROPHÉES
TROPHÉES
TROPHÉES
Activité
© 2026 SkillForge. Forgez votre futur.
CGU - Mobile
KROMA
Acceptation des conditions
Acceptation des conditions
Acceptation des conditions
Acceptation des conditions
Acceptation des conditions
© 2026 SkillForge. Forgez votre futur.
Authentification - Mobile
KROMA
Connexion
Connexion
© 2026 SkillForge. Forgez votre futur.
Confidentialité - Mobile
KROMA
Confidentialité
Confidentialité
Confidentialité
Confidentialité
Confidentialité
© 2026 SkillForge. Forgez votre futur.
About - Mobile
KROMA
Comment ça marche ?
Marche
lectus vitae interdum. Interdum et malesuada fames ac ante ipsum primis in faucibus. Nunc lobortis tortor augue, eget eleifend augue consectetur fermentum.
Marche
lectus vitae interdum. Interdum et malesuada fames ac ante ipsum primis in faucibus. Nunc lobortis tortor augue, eget eleifend augue consectetur fermentum.
© 2026 SkillForge. Forgez votre futur.
Admin - Mobile
KROMA
Créer un quiz
Ajouter une question
Publier le Quiz
© 2026 SkillForge. Forgez votre futur.
Social - Mobile
KROMA
AMI
Ami
Actualité
AMI
Ami
Actualité
AMI
Ami
Actualité
AMI
Ami
Actualité
© 2026 SkillForge. Forgez votre futur.

Frontend

La partie visible de l'application, celle avec laquelle l'utilisateur interagit, a été développée avec React. Ce choix s'explique par la nécessité de créer une interface dynamique et réactive, comparable à une application mobile native.

Stack Technique :

- **Vite :** J'ai utilisé Vite pour initialiser le projet. Contrairement à des outils plus anciens (comme Webpack), Vite offre un démarrage presque instantané du serveur de développement grâce aux modules ES natifs.
- **TypeScript :** Le projet est entièrement typé. Cela permet d'éviter de nombreuses erreurs courantes (comme essayer d'accéder à une propriété qui n'existe pas sur un objet) et améliore la maintenance du code.
- **Tailwind CSS :** Pour le style, j'ai utilisé Tailwind. Au lieu d'écrire des fichiers CSS séparés, j'applique des classes directement dans le HTML (TSX). Cela permet un développement très rapide et une cohérence visuelle immédiate.

Architecture du code

J'ai organisé mon dossier source (src) de manière modulaire pour le rendre maintenable :

/components : Contient les briques d'interface réutilisables (Boutons, Cartes, Champs de formulaire).

Exemple : Le composant Button.tsx est utilisé partout, garantissant que tous les boutons ont le même style.

/pages : Chaque fichier ici correspond à une "page" de l'application (Accueil, Login, Jeu...).

/contexts : Pour la gestion d'état global (ex: savoir si l'utilisateur est connecté).

/services : Contient la logique d'appel à l'API (les requêtes HTTP).

Fonctionnalités Clés et Implémentation

A. Le Système d'Authentification

La sécurité commence dès l'entrée. J'ai mis en place des formulaires d'inscription (**Register.tsx**) et de connexion (**Login.tsx**).

Validation : Avant même d'envoyer les données au serveur, je vérifie côté client que l'email est valide et que le mot de passe est assez complexe.

Feedback : Si la connexion échoue (ex: mauvais mot de passe), une notification (Toast) informe l'utilisateur grâce à la librairie sonner.

B. Le Moteur de Jeu (Quiz)

C'est le cœur du projet. J'ai développé un composant complexe pour gérer le déroulement d'un quiz (**ChallengePlay.tsx**).

Logique :

1. L'utilisateur lance un quiz -> Appel API pour vérifier s'il a assez de cœurs.
2. Les questions s'affichent une par une.
3. À chaque réponse, un feedback visuel (Vert/Rouge) indique immédiatement si c'est juste ou faux.
4. À la fin, un écran de résumé affiche le score et les récompenses gagnées (XP, Pièces).

Gestion d'état : J'ai dû gérer de nombreuses variables (question actuelle, score, réponses sélectionnées). Pour éviter de complexifier le composant, j'ai utilisé des Hooks React pour isoler cette logique.

C. L'Interface Utilisateur

L'objectif était de faire "premium".

Design Responsive : Grâce aux classes préfixées de Tailwind (md:, lg:), la mise en page change automatiquement selon l'écran. Sur mobile, le menu passe en bas, pour être accessible au pouce.

Mode Sombre : L'interface utilise des teintes sombres pour réduire la fatigue visuelle et donner un aspect moderne.

Difficultés rencontrées et Solutions

Problème : Au début, je ne savais pas comment conserver la connexion de l'utilisateur quand il rafraîchissait la page.

Solution : J'ai mis en place un contexte qui vérifie au chargement de l'application si une session est active via l'API, et restaure l'utilisateur le cas échéant.

Cette partie Frontend m'a permis de comprendre l'importance de découper son code en petits composants autonomes plutôt que de faire des pages géantes impossibles à maintenir.

Backend

Le moteur. C'est lui qui traite les règles du jeu, sécurise les données et communique avec la base de données.

Stack Technique :

- **Node.js & Express :** J'ai choisi cet environnement car il permet d'utiliser le même langage qu'au Frontend. Cela simplifie le "Context Switching" pour un développeur Full Stack.
- **Prisma ORM :** Plutôt que d'écrire des requêtes SQL brutes (sensibles aux injections et dures à maintenir), j'ai utilisé Prisma. Il me permet de manipuler ma base de données comme des objets classiques.
- **PostgreSQL :** Une base de données relationnelle robuste, parfaite pour structurer les données utilisateurs et leurs relations (Score, Amis, Historique).

Architecture du code

J'ai structuré mon serveur selon le modèle MVC (adapté) :

Routes : Définissent les URLs accessibles (ex: GET /api/challenges).

Controllers : Reçoivent la requête, vérifient les paramètres, et renvoient la réponse (HTTP 200, 400, 500...).

Services : Contiennent la logique métier. C'est là que je calcule les points, vérifie les réponses, etc.

Points Techniques Majeurs

A. Sécurisation des Routes

Toutes les routes sensibles (jouer, voir son profil) sont protégées. J'ai créé un **middleware d'authentification**.

- À chaque requête, ce middleware vérifie la présence d'un Token de session valide.
- Si le token est absent ou expiré, le serveur renvoie une erreur 401 Unauthorized avant même que le contrôleur ne soit exécuté.

B. Gestion de la Progression

C'est un point critique : quand un utilisateur finit un quiz, plusieurs choses doivent se passer en même temps :

1. Enregistrer le résultat dans l'historique.
2. Ajouter l'XP au profil du joueur.
3. Ajouter les pièces au portefeuille du joueur.

Si le serveur plante entre l'étape 2 et 3, le joueur a son XP mais pas ses pièces -> Incohérence.

Solution : J'ai utilisé les Transactions Prisma. Soit tout réussit, soit tout est annulé. Cela garantit que les données restent toujours intègres.

C. Validation des Données

On ne peut jamais faire confiance à ce que le Frontend envoie (un utilisateur malveillant peut modifier la requête). J'utilise la librairie **Zod** pour valider strictement les données entrantes.

- Exemple : Si j'attends un email, Zod vérifie que c'est bien une chaîne de caractères au format email. Sinon, la requête est rejetée immédiatement.

Base de Données

Mon schéma Prisma (**schema.prisma**) définit clairement les modèles.

- Le modèle User est central.
- Les relations sont définies avec des clés étrangères qui assurent la cohérence référentielle (on ne peut pas avoir un score lié à un utilisateur qui n'existe pas).

Sécurité Avancée

J'ai veillé à implémenter les bonnes pratiques OWASP :

- **CORS :** Configuré pour n'accepter que les requêtes venant de mon Frontend.
- **Rate Limiting :** Limite le nombre de requêtes par IP pour éviter les abus.
- **Helmet :** Ajoute des headers HTTP de sécurité pour protéger contre certaines vulnérabilités web classiques (XSS, Sniffing).

Ce développement Backend m'a appris la rigueur. Contrairement au Frontend où un bug est "visuel", un bug Backend peut corrompre des données ou créer des failles de sécurité.

Stratégie de Test et Qualité

La qualité d'une application ne se mesure pas seulement à ses fonctionnalités, mais à sa stabilité et à sa conformité aux standards.

A. Outils de Qualité de Code

Avant même de tester l'application manuellement, j'ai mis en place des outils pour détecter les erreurs automatiquement pendant l'écriture du code :

- **TypeScript :** Comme mentionné, le typage statique empêche environ 15% des bugs courants.
- **ESLint :** Pour forcer des règles de code cohérentes.
- **Prettier :** Pour formater automatiquement le code et le rendre lisible par n'importe quel autre développeur.

B. Tests Automatisés

Pour garantir la stabilité des fonctionnalités critiques, j'ai mis en place une suite de tests automatisés avec **Vitest** et **React Testing Library**. Ces tests simulent le parcours utilisateur de manière fiable et rapide.

Scénarios couverts :

1. Authentification (auth.test.tsx) :

- Vérification du formulaire d'inscription.
- Simulation de réussite (Appel API mocké + Redirection).
- Gestion des erreurs (Affichage du Toast si adresse e-mail déjà prise).

2. Mécanique de Jeu (game.test.tsx) :

- Test unitaire du composant QuizInterface
- Validation du feedback visuel immédiat (Vert pour bonne réponse, Rouge pour mauvaise).

3. **Boutique (shop.test.tsx) :**

- Validation de l'achat de cœurs (Mock du service User).
- Blocage du bouton d'achat si le solde de pièces est insuffisant.

```
> skill-forg@0.0.0 test
> vitest


 DEV  v4.0.18 C:/Users/cnkkf/Desktop/GitProject/skill-forg

 ✓ src/tests/game.test.tsx (2 tests) 24ms
 ✓ src/tests/auth.test.tsx (1 test) 99ms
 ✓ src/tests/shop.test.tsx (2 tests) 41ms

 Test Files  3 passed (3)
      Tests  5 passed (5)
   Start at  10:09:02
   Duration  897ms (transform 211ms, setup 202ms, import 625ms, tests 164ms, environment 1.19s)

 PASS  Waiting for file changes...
       press h to show help, press q to quit
```

C. Performance

J'ai utilisé l'outil Lighthouse de Google Chrome pour auditer la performance du Frontend.

Performance :

- **Mobile :** 96/100
- **Desktop :** 100/100

Accessibilité :

- **Mobile :** 95/100
- **Desktop :** 95 / 100

Bonnes pratiques :

- **Mobile :** 100/100
- **Desktop :** 100/ 100

SEO :

- **Mobile :** 100/100
- **Desktop :** 100/ 100

Rapport du 29 janv. 2026, 10:17:54

https://skill-forge-pi-ten.vercel.app/

Analyser

Mobile Bureau

Découvrez l'expérience de vos utilisateurs

Aucune donnée

Analysez les problèmes de performances

Performances	Accessibilité	Bonnes pratiques	SEO
96	95	100	100

Rapport du 29 janv. 2026, 10:17:54

https://skill-forge-pi-ten.vercel.app/

Analyser

Mobile | Bureau

Découvrez l'expérience de vos utilisateurs

Aucune donnée

Analysez les problèmes de performances

Performances	Accessibilité	Bonnes pratiques	SEO
100	95	100	100

D. Gestion des Erreurs

Une bonne application doit savoir gérer l'imprévu.

Côté Backend : J'ai un middleware d'erreur global. Si une route plante, le serveur ne crash pas ; il renvoie une erreur 500 propre avec un message générique pour l'utilisateur, tout en loggant l'erreur détaillée dans la console serveur.

Côté Frontend : J'utilise des "Error Boundaries" pour afficher une page "Oups, une erreur est survenue" au lieu d'une page blanche si un composant React plante.

Cette phase de test m'a permis de corriger plusieurs petits bugs d'affichage sur mobile et de sécuriser des cas que je n'avais pas prévus au départ.

Mise en Production

Pour rendre l'application accessible sur le web, j'ai dû déployer le Frontend et le Backend sur des serveurs distants.

A. Déploiement Frontend (Vercel)

J'ai choisi **Vercel** pour le Frontend car il est optimisé pour les projets React/Vite.

Le déploiement est continu (CD) : Chaque fois que je push mon code sur GitHub, Vercel détecte le changement, lance le npm run build, et met à jour le site en quelques secondes.

B. Déploiement Backend (Render + Neon)

Pour le Backend, j'ai opté pour une architecture serverless/managed plus moderne et économique :

1. Render (Hébergement API) : Plateforme de déploiement similaire à Vercel mais pour les backends Node.js.

- Déploiement automatique depuis GitHub (CD).
- Gestion automatique du HTTPS, des logs, et des redémarrages.
- Mise en veille automatique sur le plan gratuit, que j'ai contré avec cron-job.org qui va venir ping mon serveur toutes les 5 minutes.

2. Neon (Base de données PostgreSQL) : Base de données serverless PostgreSQL.

- Pas besoin de gérer un serveur de base de données.
- Connexion sécurisée via URL cryptée.
- Scaling automatique selon la charge.

Bilan du Projet

A. Compétences Acquises

Ce projet KROMA a été une expérience formatrice intense. J'ai pu valider l'ensemble des compétences du titre DWWM :

- Maquetter une application.
- Développer une interface web responsive et dynamique (React).
- Créer une base de données relationnelle (PostgreSQL).
- Développer des composants d'accès aux données (API Node/Express).

B. Difficultés et Solutions

La plus grande difficulté a été de gérer la synchronisation des données (ex: quand on perd une vie, cela doit se voir tout de suite dans la barre de navigation). J'ai résolu cela en apprenant à bien gérer mes contextes React, pour avoir une "Vérité unique" pour les données utilisateur.

C. Axes d'Amélioration

Si je devais continuer ce projet demain :

- **Mode Multijoueur :** Permettre à deux joueurs de s'affronter en temps réel (via WebSockets / Socket.io).
- **Application Mobile :** Transformer le site en vraie application mobile (PWA ou React Native) pour être sur les Stores.
- **IA Générative :** Utiliser l'IA pour générer automatiquement des questions de quiz à partir d'un thème donné, pour ne jamais manquer de contenu.

Conclusion Générale

Le projet KROMA est fonctionnel et répond au cahier des charges initial. Il représente une base solide et évolutive. Au-delà du code, ce projet m'a appris à gérer mon temps, à faire des choix technologiques pragmatiques et à ne jamais négliger l'expérience utilisateur. Je suis fier de présenter ce travail qui concrétise ma transition vers le métier de Développeur Web.